SUPPORT FORMATEUR · ALGORITHMIQUE PSEUDO-CODÉE

POO PYTHON · B1 DÉVELOPPEMENT

TOUS LES ALGORITHMES DU JEU

Gandalf contre le Roi Mort d'Angmar
Du premier objet à CombatManager.start()

GANDALF
3 attaques · PV · XP

VS

ROI MORT D'ANGMAR
3 attaques · esquive · XP

4 h 30 · pauses comprises

6 versions progressives + TP

Pseudo-code prêt à projeter

Correspondance contrôlée avec le pack « CODE_PROGRESSIF_POO_PYTHON_COMBAT_MANAGER_V2 » et la conduite formateur V2.

GUIDE D'UTILISATION

Présenter la logique avant le code

Ce document ne remplace pas la conduite orale V2. Il donne le pseudo-code de toutes les constructions successives, dans le même ordre que les dossiers exécutables. Présenter l'algorithme, demander aux étudiants de prédire son résultat, puis le traduire dans VS Code.

Règle de lecture Les versions V1, V2, V3 d'une même méthode sont des évolutions successives. Ne jamais essayer de les exécuter simultanément. La version finale est celle du dossier `06_jeu_final`.

Repères horaires - formation de 270 minutes

Créneau	Durée	Contenu
09:00 - 09:15	15 min	Concevoir les responsabilités (sans coder)
09:15 - 09:50	35 min	01 · Classes et démarrage
09:50 - 10:25	35 min	02 · Attaques simples
10:25 - 10:35	10 min	Pause
10:35 - 11:10	35 min	03 · Encapsulation et esquive
11:10 - 11:40	30 min	04 · Expérience et multiplicateur
11:40 - 12:10	30 min	05 · Héritage et polymorphisme
12:10 - 12:20	10 min	Pause
12:20 - 13:00	40 min	06 · CombatManager.start() complet
13:00 - 13:20	20 min	TP et vérifications
13:20 - 13:30	10 min	Bilan et correction

Conventions

`<-` affecte une valeur ; `ECRIRE` affiche ; `LIRE` attend une saisie ; `RETOURNER` termine une méthode et donne un résultat ; `APPELER` exécute une méthode.

Structures

`SI / SINON / FIN SI`, `TANT QUE`, `POUR CHAQUE`, `SORTIR DE LA BOUCLE`, `LEVER ERREUR`. Les noms de classes et méthodes reproduisent le projet Python.

Les responsabilités à annoncer au tableau

Personnage

État, PV, XP, dégâts, esquive, attaque, coefficient.

Gandalf / RoiMortAngmar

Catalogues des trois coups et présentations spécialisées.

CombatManager

Choix utilisateur, adversaire automatique, tours, affichage, victoire.

main.py

Créer CombatManager puis appeler start(). Rien d'autre.

AVANT L'ETAPE 01 · 09:00 - 09:15

Contrat du jeu et calculs à faire prédire

À lire avant de programmer : les deux personnages commencent à 100 PV et 0 XP. Le joueur contrôle Gandalf, l'ordinateur contrôle le Roi Mort. Chaque personnage possède exactement trois attaques.

Personnage	Attaque	Dégâts de base	Probabilité d'esquive de la cible
Gandalf	Coup de bâton	12	10 %
Gandalf	Éclair magique	20	25 %
Gandalf	Flamme d'Anor	32	50 %
Roi Mort	Coup d'épée	14	10 %
Roi Mort	Lame de Morgul	22	30 %
Roi Mort	Souffle d'effroi	30	45 %

```
COEFF = min(2 ; 1 + XP / 100)
DÉGÂTS = int(dégâts_de_base × COEFF)
```

Esquive

Si le tirage aléatoire est strictement inférieur au taux d'esquive : aucun PV retiré, aucun XP gagné.

Réussite

Calculer les dégâts avec les XP actuels ; retirer les PV ; seulement ensuite attribuer 10 XP.

Question orale Gandalf a 20 XP et utilise Coup de bâton. Combien de dégâts ? **int(12 × 1,20) = 14.** Si le coup est esquivé : **0 dégât et 0 XP supplémentaire.**

Précisions importantes La probabilité dépend du coup choisi, pas d'une statistique fixe du défenseur. L'XP totale peut dépasser 100 ; seul le coefficient est limité à 2. Un personnage à 0 PV ne peut plus agir.

ÉTAPE 01 · 09:15 - 09:50 · 35 min

Créer les classes et démarrer une démonstration

À ce stade, les deux combattants existent. Ils ne se battent pas encore. Le gestionnaire crée les objets et affiche leur état.

`Dossier de référence : etapes/01_classes`

Notions à rappeler

- Une classe est un modèle ; une instance est un objet concret.
- Un constructeur initialise l'état. self désigne l'instance concernée.
- La lecture des PV passe par une propriété. Une méthode peut retourner une valeur ou une phrase.

A01 Point d'entrée - main.py

MÉTHODE	ENTRÉES	PRODUIT
main.py	aucune	démarre le gestionnaire

La seule responsabilité de main.py est de lancer le jeu.

```
ALGORITHME POINT_ENTREE
DEBUT
    combat_manager <- NOUVEAU CombatManager()
    APPELER combat_manager.start()
FIN
```

À vérifier main.py ne choisit pas les coups et ne contient aucune boucle de combat.

A02 Construire un personnage

MÉTHODE	ENTRÉES	PRODUIT
Personnage.__init__	nom, points_de_vie	un objet initialisé

Initialiser deux attributs propres à chaque instance.

```
METHODE Personnage.__init__(nom, points_de_vie)
DEBUT
    self.nom <- nom
    self._points_de_vie <- points_de_vie
FIN
```

À vérifier Gandalf et le Roi Mort ont chacun leur propre compteur de PV.

A03 Consulter les points de vie

MÉTHODE	ENTRÉES	PRODUIT
Personnage.points_de_vie	aucune	les PV actuels

Dans le code Python, l'accesseur est décoré avec @property.

```
PROPRIETE points_de_vie
DEBUT
    RETOURNER self._points_de_vie
FIN
```

À vérifier Lire personnage.points_de_vie, sans modifier directement _points_de_vie.

A04 Déterminer si un personnage vit

MÉTHODE	ENTRÉES	PRODUIT
Personnage.est_vivant	aucune	vrai ou faux

Une condition booléenne résume un état.

```
METHODE est_vivant()
DEBUT
    RETOURNER self._points_de_vie > 0
FIN
```

À vérifier Pour 100 PV : VRAI ; pour 0 PV : FAUX.

A05 Présenter un personnage

MÉTHODE	ENTRÉES	PRODUIT
Personnage.presentation	aucune	un texte

Une méthode peut fabriquer une chaîne de caractères.

```
METHODE presentation()
DEBUT
    RETOURNER self.nom + " entre dans l'arène."
FIN
```

À vérifier La méthode retourne le texte ; CombatManager décide de l'afficher.

A06 Créer le gestionnaire

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.__init__	aucune	deux attributs objets

Le gestionnaire possède une référence vers chaque combattant.

```
METHODE CombatManager.__init__()
DEBUT
    self.gandalf <- NOUVEAU Personnage("Gandalf", 100)
    self.roi <- NOUVEAU Personnage("Roi Mort d'Angmar", 100)
FIN
```

À vérifier Il n'y a pas encore de classes Gandalf et RoiMortAngmar distinctes.

A07 Démonstration de démarrage

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.start	aucune	des textes affichés

La méthode start() est pour l'instant un scénario de présentation.

```
METHODE CombatManager.start()
DEBUT
    ECRIRE self.gandalf.presentation()
    ECRIRE self.roi.presentation()
    ECRIRE "PV de Gandalf : ", self.gandalf.points_de_vie
    ECRIRE "Les deux vivent : ",
           self.gandalf.est_vivant() ET self.roi.est_vivant()
FIN
```

À vérifier On observe deux personnages vivants avec 100 PV.

ÉTAPE 02 · 09:50 - 10:25 · 35 min

Créer les trois attaques et faire collaborer les objets

On ajoute un catalogue de coups à chaque personnage. Les premières attaques infligent leurs dégâts de base sans esquive et sans XP : le hasard sera introduit à l'étape suivante.

`Dossier de référence : etapes/02_attaques`

Notions à rappeler

- Une liste regroupe des attaques ; chaque attaque est un dictionnaire de trois informations.
- L'objet attaquant sélectionne le coup ; l'objet cible modifie lui-même ses PV.
- La méthode reçoit une référence vers un autre objet : c'est une collaboration entre instances.

A08 Décrire le catalogue des attaques

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.__init__	aucune	deux listes de trois dictionnaires

Données à projeter et recopier lors de la création des deux personnages.

```
attaques_gandalf <- [
  {nom: "Coup de bâton", degats: 12, esquive: 0.10},
  {nom: "Éclair magique", degats: 20, esquive: 0.25},
  {nom: "Flamme d'Anor", degats: 32, esquive: 0.50}
]
attaques_roi <- [
  {nom: "Coup d'épée", degats: 14, esquive: 0.10},
  {nom: "Lame de Morgul", degats: 22, esquive: 0.30},
  {nom: "Souffle d'effroi", degats: 30, esquive: 0.45}
]
self.gandalf <- NOUVEAU Personnage("Gandalf", 100, attaques_gandalf)
self.roi <- NOUVEAU Personnage("Roi Mort d'Angmar", 100, attaques_roi)
```

À vérifier Les indices internes de la liste sont 0, 1 et 2. Pour l'instant, la clé esquive est stockée mais non exploitée.

A09 Accepter un catalogue facultatif

MÉTHODE	ENTRÉES	PRODUIT
Personnage.__init__ - évolution	nom, points_de_vie, attaques facultatives	objet initialisé avec une liste

L'argument facultatif évite de rendre le catalogue obligatoire.

```
METHODE Personnage.__init__(nom, points_de_vie, attaques = ABSENT)
DEBUT
    self.nom <- nom
    self._points_de_vie <- points_de_vie
    SI attaques EST ABSENT ALORS
        self.attaques <- LISTE_VIDE()
    SINON
        self.attaques <- attaques
    FIN SI
FIN
```

À vérifier En Python, la V2 utilise attaques=None puis crée [] si nécessaire ; elle n'utilise pas [] comme valeur par défaut.

A10 Retirer les PV sans passer sous zéro

MÉTHODE	ENTRÉES	PRODUIT
Personnage.subir_degats - V1	valeur	PV mis à jour

Un seul point du programme porte la règle de diminution des PV.

```
METHODE subir_degats(valeur)
DEBUT
    self._points_de_vie <- MAXIMUM(0, self._points_de_vie - valeur)
FIN
```

À vérifier Avec 8 PV et 12 dégâts, les PV deviennent 0, jamais -4.

A11 Attaquer sans esquive ni XP

MÉTHODE	ENTRÉES	PRODUIT
Personnage.attaquer - V1	cible, index_attaque	message de résultat

Première version volontairement simple pour isoler les appels de méthodes.

```
METHODE attaquer(cible, index_attaque)
DEBUT
    attaque <- self.attaques[index_attaque]
    APPELER cible.subir_degats(attaque["degats"])
    RETOURNER self.nom + " utilise " + attaque["nom"]
              + " : " + attaque["degats"] + " dégâts."
FIN
```

À vérifier Un Éclair magique (index 1) inflige 20 PV à un ennemi à 100 PV : il lui reste 80 PV.

A12 Démo de deux frappes contrôlées

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.start - étape 02	aucune	PV après deux attaques

La console montre la modification de deux objets différents.

```
METHODE CombatManager.start()
DEBUT
    ECRIRE self.gandalf.attaquer(self.roi, 1)
    ECRIRE "PV du Roi Mort : ", self.roi.points_de_vie
    ECRIRE self.roi.attaquer(self.gandalf, 0)
    ECRIRE "PV de Gandalf : ", self.gandalf.points_de_vie
FIN
```

À vérifier Résultat attendu : Roi Mort = 80 PV ; Gandalf = 86 PV.

ÉTAPE 03 · 10:35 - 11:10 · 35 min

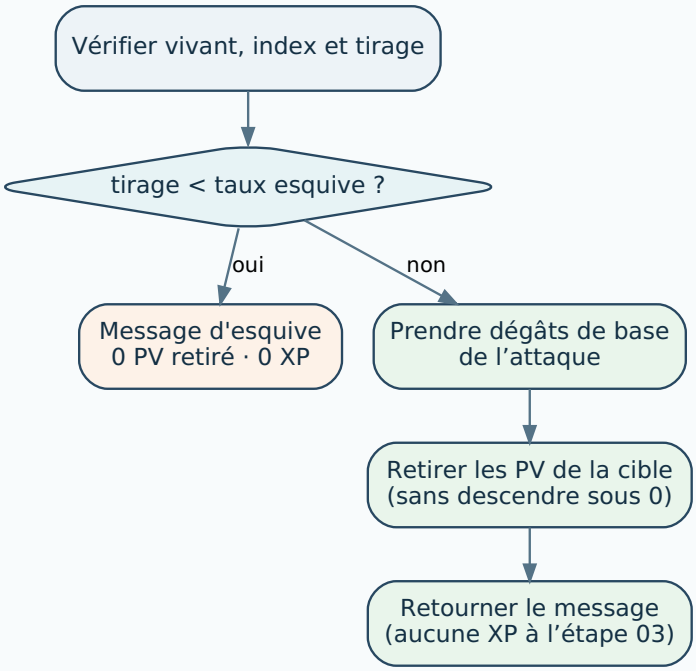
Protéger l'état et introduire l'esquive

À présent une attaque peut manquer sa cible. On valide aussi les données : PV positifs à la création, dégâts non négatifs, attaque valide et combattants vivants.

`Dossier de référence : etapes/03_esquive`

Notions à rappeler

- Encapsulation : les PV sont modifiés par subir_degats(), pas par le gestionnaire.
- Un tirage aléatoire réel appartient à [0 ; 1[, mais on peut injecter un tirage fixe pour tester.
- La probabilité d'esquive dépend de l'attaque choisie ; elle est calculée automatiquement, sans action « esquiver ».



Logique de l'étape 03 : test d'esquive puis dégâts de base. Le gain d'XP et le coefficient n'apparaissent qu'à l'étape 04.

A13 Refuser les PV initiaux invalides

MÉTHODE	ENTRÉES	PRODUIT
Personnage.__init__ - ajout	points_de_vie	erreur ou initialisation

La vérification intervient avant l'affectation des attributs.

```
AU DEBUT DE __init__(nom, points_de_vie, attaques = ABSENT)
    SI points_de_vie <= 0 ALORS
        LEVER ERREUR Valeur("Les PV doivent être positifs")
    FIN SI
    ... puis exécuter les affectations de A09
```

À vérifier Un personnage ne peut pas être créé avec 0 PV ou des PV négatifs.

A14 Contrôler les dégâts reçus

MÉTHODE	ENTRÉES	PRODUIT
Personnage.subir_degats - V2	valeur	erreur ou PV mis à jour

La validation interdit qu'une valeur négative puisse soigner un personnage par accident.

```
METHODE subir_degats(valeur)
DEBUT
    SI valeur < 0 ALORS
        LEVER ERREUR Valeur("Dégâts négatifs interdits")
    FIN SI
    self._points_de_vie <- MAXIMUM(0, self._points_de_vie - valeur)
FIN
```

À vérifier Cette V2 conserve la borne inférieure 0 PV.

A15 Choisir entre esquive et réussite

MÉTHODE	ENTRÉES	PRODUIT
Personnage.attaquer - V2	cible, index_attaque, tirage facultatif	message et éventuellement dégâts

L'ordre des vérifications est important ; aucun dégât n'est infligé en cas d'erreur ou d'esquive.

```
METHODE attaquer(cible, index_attaque, tirage = ABSENT)
DEBUT
    SI NON self.est_vivant() OU NON cible.est_vivant() ALORS
        LEVER ERREUR Valeur("Combattant déjà vaincu")
    FIN SI
    SI index_attaque < 0 OU index_attaque >= TAILLE(self.attaques) ALORS
        LEVER ERREUR Index("Attaque inexistante")
    FIN SI
    attaque <- self.attaques[index_attaque]
    SI tirage EST ABSENT ALORS
        tirage <- ALEATOIRE_REEL_DANS_[0,1[()
    FIN SI
    SI tirage < 0 OU tirage >= 1 ALORS
        LEVER ERREUR Valeur("Tirage invalide")
    FIN SI
    SI tirage < attaque["esquive"] ALORS
        RETOURNER cible.nom + " esquive " + attaque["nom"]
    FIN SI
    APPELER cible.subir_degats(attaque["degats"])
    RETOURNER self.nom + " réussit " + attaque["nom"]
FIN
```

À vérifier Une attaque de taux 0,10 est esquivée avec tirage=0,00 ; elle touche avec tirage=0,99. À ce stade, il n'y a toujours pas d'XP.

A16 Démontrer les deux branches

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.start - étape 03	aucune	deux états de PV observables

On fixe le tirage pour ne jamais dépendre du hasard pendant la démonstration.

```
METHODE CombatManager.start()
DEBUT
    ECRIRE gandalf.attaquer(roi, 0, tirage = 0.00)
    ECRIRE "PV après esquive : ", roi.points_de_vie
    ECRIRE gandalf.attaquer(roi, 0, tirage = 0.99)
    ECRIRE "PV après réussite : ", roi.points_de_vie
FIN
```

À vérifier PV observés : 100 puis 88.

ÉTAPE 04 · 11:10 - 11:40 · 30 min

Ajouter l'expérience et le bonus de puissance

Chaque coup réussi rapporte 10 XP. Le bonus de puissance dépend des XP acquis AVANT le coup ; il ne peut pas dépasser ×2.

`Dossier de référence : etapes/04_xp`

Notions à rappeler

- L'XP est un attribut d'instance (_xp) initialisé à 0.
- La règle du coefficient est COEFF = min(2 ; 1 + XP / 100).
- L'ordre exact est : calcul du coefficient, dégâts, retrait des PV, puis gain de 10 XP.

A17 Initialiser et exposer l'expérience

MÉTHODE	ENTRÉES	PRODUIT
Personnage.__init__ + propriété xp	aucune	XP lisibles à l'extérieur

L'XP ne peut être augmentée que par la méthode du personnage.

```
DANS __init__ APRES L'INITIALISATION DES PV
    self._xp <- 0

PROPRIETE xp
DEBUT
    RETOURNER self._xp
FIN
```

À vérifier À l'entrée dans l'arène, Gandalf et le Roi Mort ont chacun 0 XP.

A18 Calculer le coefficient

MÉTHODE	ENTRÉES	PRODUIT
Personnage.coefficient_puissance	aucune	un réel entre 1 et 2

La formule reste la même pour les deux combattants.

```
METHODE coefficient_puissance()
DEBUT
    coefficient <- 1 + self._xp / 100
    RETOURNER MINIMUM(2.0, coefficient)
FIN
```

À vérifier 0 XP → ×1,00 ; 10 XP → ×1,10 ; 50 XP → ×1,50 ; 150 XP → ×2,00.

A19 Attribuer l'expérience

MÉTHODE	ENTRÉES	PRODUIT
Personnage.gagner_xp	aucune	XP augmentée de 10

Cette méthode n'est appelée qu'après un coup réussi.

```
METHODE gagner_xp()
DEBUT
    self._xp <- self._xp + 10
FIN
```

À vérifier Une attaque esquivée ne passe jamais par cette méthode.

A20 Attaquer avec un bonus

MÉTHODE	ENTRÉES	PRODUIT
Personnage.attaquer - V3	cible, index_attaque, tirage facultatif	message, PV, XP

Conserver toutes les validations et le test d'esquive de A15, puis remplacer uniquement le traitement d'une attaque réussie.

```
METHODE attaquer(cible, index_attaque, tirage = ABSENT)
DEBUT
    SI NON self.est_vivant() ALORS
        LEVER ERREUR Valeur("Attaquant vaincu")
    FIN SI
    SI NON cible.est_vivant() ALORS
        LEVER ERREUR Valeur("Cible vaincue")
    FIN SI
    SI index_attaque < 0 OU index_attaque >= TAILLE(self.attaques) ALORS
        LEVER ERREUR Index("Attaque inexistante")
    FIN SI
    attaque <- self.attaques[index_attaque]
    SI tirage EST ABSENT ALORS
        tirage <- ALEATOIRE_REEL_DANS_[0,1[()
    FIN SI
    SI tirage < 0 OU tirage >= 1 ALORS
        LEVER ERREUR Valeur("Tirage invalide")
    FIN SI
    SI tirage < attaque["esquive"] ALORS
        RETOURNER le message d'esquive       // 0 dégât, 0 XP
    FIN SI

    coefficient <- self.coefficient_puissance()  // XP avant coup
    degats <- ENTIER_INFERIEUR(attaque["degats"] * coefficient)
    APPELER cible.subir_degats(degats)
    APPELER self.gagner_xp()                     // +10 XP après coup
    RETOURNER le message de dégâts et de gain d'XP
FIN
```

À vérifier ENTIER_INFERIEUR correspond ici à int() de Python pour une valeur positive. Ne jamais créditer l'XP avant le calcul des dégâts.

A21 Tracer deux attaques réussies

MÉTHODE
CombatManager.start - étape 04

ENTRÉES
aucune

PRODUIT
PV et XP après deux coups

Rendre visible le décalage entre XP gagnée et puissance du coup suivant.

```
METHODE CombatManager.start()
DEBUT
    ECRIRE gandalf.attaquer(roi, 0, tirage = 0.99)
    ECRIRE "XP : ", gandalf.xp, " | PV cible : ", roi.points_de_vie
    ECRIRE gandalf.attaquer(roi, 0, tirage = 0.99)
    ECRIRE "XP : ", gandalf.xp, " | PV cible : ", roi.points_de_vie
FIN
```

À vérifier Premier coup : 12 dégâts, 10 XP, roi à 88 PV. Deuxième : int(12 × 1,1) = 13 dégâts, 20 XP, roi à 75 PV.

Trace à écrire au tableau : trois états successifs

Action de Gandalf	XP avant	Dégâts	XP après	PV du Roi Mort
Esquive (tirage 0,00)	0	0	0	100
Réussite (0,99)	0	12	10	88
Réussite (0,99)	10	13	20	75

ÉTAPE 05 · 11:40 - 12:10 · 30 min

Spécialiser les combattants avec l'héritage

On déplace les catalogues de coups vers les classes Gandalf et RoiMortAngmar. Le code commun demeure dans Personnage ; le gestionnaire manipule les deux types via la même interface.

`Dossier de référence : etapes/05_heritage`

Notions à rappeler

- Gandalf et RoiMortAngmar héritent de Personnage : ils réutilisent toutes ses méthodes.
- super().__init__ construit l'état hérité avec des valeurs spécifiques.
- La redéfinition de presentation() illustre le polymorphisme sans dupliquer la logique d'attaque.

A22 Initialiser Gandalf

MÉTHODE	ENTRÉES	PRODUIT
Gandalf.__init__	aucune	Gandalf avec 100 PV, 0 XP et trois coups

Données propres à la sous-classe ; le constructeur de la classe mère réalise l'initialisation.

```
CLASSE Gandalf HERITE DE Personnage
METHODE __init__()
DEBUT
    attaques <- [
      {nom: "Coup de bâton", degats: 12, esquive: 0.10},
      {nom: "Éclair magique", degats: 20, esquive: 0.25},
      {nom: "Flamme d'Anor", degats: 32, esquive: 0.50}
    ]
    APPELER super().__init__("Gandalf", 100, attaques)
FIN
```

À vérifier Ne pas dupliquer subir_degats(), gagner_xp() ni attaquer() dans cette sous-classe.

A23 Initialiser le Roi Mort

MÉTHODE	ENTRÉES	PRODUIT
RoiMortAngmar.__init__	aucune	Roi Mort avec 100 PV, 0 XP et trois coups

Même contrat, autre paramétrage.

```
CLASSE RoiMortAngmar HERITE DE Personnage
METHODE __init__()
DEBUT
    attaques <- [
      {nom: "Coup d'épée", degats: 14, esquive: 0.10},
      {nom: "Lame de Morgul", degats: 22, esquive: 0.30},
      {nom: "Souffle d'effroi", degats: 30, esquive: 0.45}
    ]
    APPELER super().__init__("Roi Mort d'Angmar", 100, attaques)
FIN
```

À vérifier On manipule les deux objets exactement comme des Personnage.

A24 Redéfinir les présentations

MÉTHODE	ENTRÉES	PRODUIT
Gandalf.presentation / RoiMortAngmar.presentation	aucune	deux phrases différentes

Le nom de la méthode est identique, mais chaque sous-classe propose son comportement.

```
METHODE Gandalf.presentation()
DEBUT
    RETOURNER "Gandalf lève son bâton. Vous jouez Gandalf !"
FIN

METHODE RoiMortAngmar.presentation()
DEBUT
    RETOURNER "Le Roi Mort d'Angmar avance dans l'ombre."
FIN
```

À vérifier C'est un exemple de polymorphisme par redéfinition de méthode.

A25 Instancier les sous-classes

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.__init__ - étape 05	aucune	deux objets spécialisés

La classe CombatManager conserve les mêmes attributs gandalf et roi.

```
METHODE CombatManager.__init__()
DEBUT
    self.gandalf <- NOUVEAU Gandalf()
    self.roi <- NOUVEAU RoiMortAngmar()
FIN
```

À vérifier La logique du combat ne dépendra pas du code spécifique des sous-classes.

A26 Faire apparaître le polymorphisme

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.start - étape 05	aucune	présentations et démonstration de frappe

La boucle appelle la même méthode sur deux objets de types différents.

```
METHODE CombatManager.start()
DEBUT
    POUR CHAQUE personnage DANS (self.gandalf, self.roi) FAIRE
        ECRIRE personnage.presentation()
        ECRIRE "Ses attaques : ", NOMS(personnage.attaques)
    FIN POUR
    ECRIRE self.gandalf.attaquer(self.roi, 1, tirage = 0.99)
    ECRIRE "PV du Roi Mort : ", self.roi.points_de_vie
FIN
```

À vérifier L'éclair magique réussit : le Roi Mort passe à 80 PV.

ÉTAPE 06 · 12:20 - 13:00 · 40 min

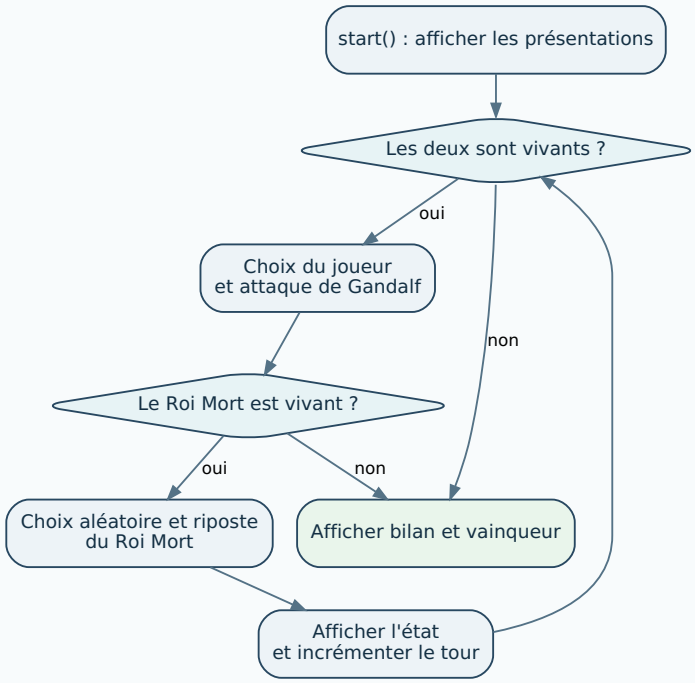
Orchestrer un combat entier avec CombatManager

C'est seulement maintenant que la démonstration devient une partie : choix du joueur, attaque automatique de l'adversaire, boucle des tours, affichage et annonce du vainqueur.

`Dossier de référence : etapes/06_jeu_final`

Notions à rappeler

- Seul CombatManager gère la console, les tours et l'ordre des actions.
- Le joueur saisit 1, 2 ou 3 ; le moteur d'attaque attend un index 0, 1 ou 2.
- Après un coup fatal de Gandalf, le Roi Mort ne riposte pas. La fin du combat est annoncée une seule fois.



Structure de la partie finale : la sortie après un coup fatal évite toute riposte.

A27 Préparer une partie

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.__init__ - final	aucune	combattants et numéro de tour

Le tour initial vaut 1.

```
METHODE CombatManager.__init__()
DEBUT
    self.gandalf <- NOUVEAU Gandalf()
    self.roi <- NOUVEAU RoiMortAngmar()
    self.tour <- 1
FIN
```

À vérifier Le constructeur ne commence pas le combat : seul start() le lance.

A28 Proposer trois choix et valider

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.choisir_attaque	saisies successives de l'utilisateur	index d'attaque 0, 1 ou 2

Une saisie incorrecte doit relancer la question, sans faire perdre le tour.

```
METHODE choisir_attaque()
DEBUT
    ECRIRE "Choisissez une attaque :"
    POUR numero DE 1 A TAILLE(self.gandalf.attaques) FAIRE
        attaque <- self.gandalf.attaques[numero - 1]
        pourcentage <- ARRONDIR(attaque["esquive"] * 100)
        ECRIRE numero, attaque["nom"], attaque["degats"], pourcentage
    FIN POUR

    TANT QUE VRAI FAIRE
        choix <- SUPPRIMER_ESPACES(LIRE("Votre choix : "))
        SI choix EST "1" OU "2" OU "3" ALORS
            RETOURNER CONVERTIR_ENTIER(choix) - 1
        SINON
            ECRIRE "Saisie invalide : tapez 1, 2 ou 3."
        FIN SI
    FIN TANT QUE
FIN
```

À vérifier Si l'utilisateur entre 0, puis abc, puis 2 : la méthode insiste puis retourne 1.

A29 Afficher l'état de deux objets

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.afficher_etat	aucune	deux lignes de synthèse

La méthode d'affichage fait partie du TP de 13:00. Elle utilise des propriétés et une méthode existantes.

```
METHODE afficher_etat()
DEBUT
    POUR CHAQUE personnage DANS (self.gandalf, self.roi) FAIRE
        ECRIRE personnage.nom, " : ",
               personnage.points_de_vie, "/",
               personnage.points_de_vie_max, " PV | ",
               personnage.xp, " XP | x",
               FORMAT_2_DECIMALES(personnage.coefficient_puissance())
    FIN POUR
FIN
```

À vérifier Exemple : Gandalf : 88/100 PV | 10 XP | x1.10.

A30 Enchaîner tous les tours

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.start - final	choix utilisateur, tirages aléatoires	vainqueur, état final

Algorithme de référence : aucune autre classe ne possède la boucle principale.

```
METHODE start()
DEBUT
    ECRIRE "=== GANDALF CONTRE LE ROI MORT D'ANGMAR ==="
    ECRIRE self.gandalf.presentation()
    ECRIRE self.roi.presentation()

    TANT QUE self.gandalf.est_vivant() ET self.roi.est_vivant() FAIRE
        ECRIRE "--- Tour ", self.tour, " ---"
        choix <- self.choisir_attaque()
        ECRIRE self.gandalf.attaquer(self.roi, choix)

        SI NON self.roi.est_vivant() ALORS
            SORTIR DE LA BOUCLE     // ne pas faire riposter un mort
        FIN SI

        choix_roi <- ENTIER_ALEATOIRE(0, TAILLE(self.roi.attaques) - 1)
        ECRIRE "Le Roi Mort choisit ", self.roi.attaques[choix_roi]["nom"]
        ECRIRE self.roi.attaquer(self.gandalf, choix_roi)
        APPELER self.afficher_etat()
        self.tour <- self.tour + 1
    FIN TANT QUE

    ECRIRE "=== FIN DU COMBAT ==="
    APPELER self.afficher_etat()
    SI self.gandalf.est_vivant() ALORS
        vainqueur <- self.gandalf
    SINON
        vainqueur <- self.roi
    FIN SI
    ECRIRE "Vainqueur : ", vainqueur.nom
FIN
```

À vérifier Si Gandalf assène un coup fatal, on quitte la boucle avant la riposte et avant l'incrément du tour.

ÉTAPE TP · 13:00 - 13:20 · 20 min

Terminer le TP et prouver les règles

Dans le dossier TP, afficher_etat() contient encore pass ; les étudiants doivent la terminer. Ils complètent aussi trois assertions sur l'esquive puis deux coups réussis.

`Dossier de référence : etapes/06_tp_a_completer puis etapes/06_jeu_final`

Notions à rappeler

- Une assertion compare un résultat réel à une valeur attendue : elle ne remplace pas l'analyse du problème.
- Le tirage forcé donne des tests reproductibles : 0,00 esquive, 0,99 touche.
- Le jeu doit aussi être testé jusqu'à sa conclusion : un gagnant, aucune riposte après le coup fatal.

A31 TP : compléter l'affichage

MÉTHODE	ENTRÉES	PRODUIT
CombatManager.afficher_etat	deux objets	état complet à l'écran

Ne pas dupliquer le calcul du coefficient : appeler la méthode des personnages.

```
POUR CHAQUE combattant DANS (gandalf, roi) FAIRE
    nom <- combattant.nom
    pv <- combattant.points_de_vie
    pv_max <- combattant.points_de_vie_max
    experience <- combattant.xp
    coeff <- combattant.coefficient_puissance()
    ECRIRE nom, pv, "/", pv_max, " PV | ", experience,
           " XP | x", FORMAT_2_DECIMALES(coeff)
FIN POUR
```

À vérifier Correction de référence : identique à A29 ; les élèves la réalisent eux-mêmes.

A32 TP : tester l'esquive et l'évolution XP

MÉTHODE	ENTRÉES	PRODUIT
test_combat.py - TP	tirages imposés	vérifications automatiques

Les valeurs attendues sont celles du pack V2 ; toutes les comparaisons doivent réussir.

```
gandalf <- NOUVEAU Gandalf()
roi <- NOUVEAU RoiMortAngmar()

APPELER gandalf.attaquer(roi, 0, tirage = 0.00)
VERIFIER roi.points_de_vie = 100
VERIFIER gandalf.xp = 0

APPELER gandalf.attaquer(roi, 0, tirage = 0.99)
VERIFIER roi.points_de_vie = 88
VERIFIER gandalf.xp = 10

APPELER gandalf.attaquer(roi, 0, tirage = 0.99)
VERIFIER roi.points_de_vie = 75
```

À vérifier Le deuxième coup réussi inflige 13 dégâts, car la première réussite a apporté 10 XP.

A33 Tests complémentaires du jeu corrigé

MÉTHODE	ENTRÉES	PRODUIT
test_combat.py - final	états et saisies contrôlés	validation de toutes les règles

Batterie de tests à présenter en synthèse ; une partie repose sur des tirages simulés et une saisie simulée.

```
VERIFIER TAILLE(Gandalf().attaques) = 3
VERIFIER TAILLE(RoiMortAngmar().attaques) = 3
VERIFIER esquive : PV inchangés ET XP inchangée
VERIFIER réussite : PV diminués ET XP +10
VERIFIER coefficient <= 2, même au-delà de 100 XP
VERIFIER subir_degats(1000) laisse 0 PV
VERIFIER attaquer une cible morte lève une erreur
VERIFIER qu'un attaquant mort ne peut pas attaquer
VERIFIER subir_degats(-5) lève une erreur
SIMULER saisies "0", "abc", "2" : index retourné = 1
SIMULER une partie sans esquive : un vainqueur est annoncé
VERIFIER qu'aucune riposte ne suit le coup fatal
```

À vérifier On peut lancer python test_combat.py dans le dossier jeu_final. Le fichier final teste aussi tour = 3 dans son scénario déterministe.

SYNTHÈSE FINALE · 13:20 - 13:30

La chaîne complète d'exécution

main.py
crée CombatManager
→
CombatManager.start()
pilote les tours
→
Personnage.attaquer()
applique les règles
→
cible.subir_degats()
retire les PV

Liste de recette à faire valider par les élèves

- ☐ main.py se contente d'instancier CombatManager et de lancer start().
- ☐ Les deux combattants ont 100 PV, 0 XP et exactement trois coups.
- ☐ Le joueur choisit 1, 2 ou 3 ; les saisies invalides sont rejetées.
- ☐ Le Roi Mort choisit au hasard l'une de ses trois attaques.
- ☐ Le test d'esquive utilise le taux du coup sélectionné.
- ☐ Une esquive produit 0 dégât et 0 XP ; un coup réussi rapporte 10 XP.
- ☐ Les XP sont attribués après le calcul du coup réussi, jamais avant.
- ☐ Le coefficient suit min(2 ; 1 + XP / 100) et les dégâts sont entiers.
- ☐ Les PV ne deviennent pas négatifs ; les dégâts négatifs sont rejetés.
- ☐ Un vaincu ne peut ni attaquer ni être attaqué ; aucune riposte après coup fatal.
- ☐ CombatManager annonce une seule victoire et permet d'observer les PV, XP et coefficients.

Repère indispensable : pour un enseignement de 4 h 30, ne pas transformer ce cours en une suite de variantes optionnelles. Le jeu final doit être achevé ; la priorité est le bon fonctionnement des méthodes A28, A30 et A31, puis les tests des règles vitales.

Correspondance pseudo-code / Python

En pseudo-code	En Python	À rappeler
APPELER manager.start()	`manager.start()`	Appel de méthode
self._xp <- 0	`self._xp = 0`	Affectation
RETOURNER condition	`return condition`	Valeur retournée
TANT QUE condition	`while condition:`	Boucle
POUR CHAQUE x DANS liste	`for x in liste:`	Parcours
SORTIR DE LA BOUCLE	`break`	Arrêt anticipé
LEVER ERREUR Valeur	`raise ValueError(...)`	Contrat de méthode
ENTIER_INFERIEUR(n)	`int(n)`	Ici les dégâts sont positifs
ALEATOIRE_REEL_DANS_[0,1[()	`random.random()`	Tirage aléatoire

Références de cohérence : conduite formateur « CombatManager 4 h 30 V2 », projet exécutable « CODE_PROGRESSIF_POO_PYTHON_COMBAT_MANAGER_V2 » et énoncé étudiant « Duel POO Python 4 h 30 ». Les algorithmes de ce recueil transcrivent les comportements effectifs des fichiers de ces versions ; ils ne proposent pas une nouvelle architecture.